Comprehensive Theory of Single-Cycle Processor Design and the ToyRISC ISA

Part 2: ToyRISC ISA Deep Dive and Instruction Encoding

Computer Architecture Laboratory – Assignment 3

February 15, 2026

Contents

1 ToyRISC ISA Overview **2**
1.1 Key Characteristics . . . 2

2 Instruction Formats – Detailed Encoding **2**
2.1 R3-Type (Register-Register-Register) . . . 2
2.1.1 Bit Extraction Formulas for R3-Type . . . 3
2.2 R2I-Type (Register-Register-Immediate) . . . 3
2.2.1 Sign Extension of 17-bit Immediate . . . 3
2.2.2 Bit Extraction Formulas for R2I-Type . . . 4
2.3 RI-Type (Register-Immediate) . . . 4
2.3.1 Bit Extraction Formulas for RI-Type . . . 4
2.4 Opcode-to-Format Mapping . . . 4

3 Detailed Instruction Semantics **5**
3.1 Arithmetic Instructions . . . 5
3.1.1 add / addi – Addition . . . 5
3.1.2 sub / subi – Subtraction . . . 5
3.1.3 mul / muli – Multiplication . . . 6
3.1.4 div / divi – Division . . . 6
3.2 Logical Instructions . . . 6
3.2.1 and / andi – Bitwise AND . . . 6
3.2.2 or / ori – Bitwise OR . . . 6
3.2.3 xor / xori – Bitwise XOR . . . 6
3.3 Comparison Instructions . . . 7
3.3.1 slt / slti – Set Less Than . . . 7
3.4 Shift Instructions . . . 7

3.4.1 sll / slli – Shift Left Logical . . . 7
3.4.2 srl / srli – Shift Right Logical . . . 7
3.4.3 sra / srai – Shift Right Arithmetic . . . 7
3.5 Memory Instructions . . . 8
3.5.1 load – Load Word from Memory . . . 8
3.5.2 store – Store Word to Memory . . . 8
3.6 Control Flow Instructions – Detailed . . . 9
3.6.1 jmp – Unconditional Jump . . . 9
3.6.2 beq – Branch if Equal . . . 9
3.6.3 bne – Branch if Not Equal . . . 9
3.6.4 blt – Branch if Less Than . . . 9
3.6.5 bgt – Branch if Greater Than . . . 9
3.7 Special Instruction: end . . . 10

4 Object File Format **10**
4.1 Structure . . . 10
4.2 Loading the Object File . . . 10

5 Assembly Language Syntax **11**
5.1 Directives . . . 11
5.2 Labels . . . 11
5.3 Operand Notation . . . 11
5.4 Instruction Ordering . . . 12

6 Complete Encoding Example **12**
6.1 Data Segment . . . 12
6.2 Instruction Encoding . . . 12
6.2.1 Instruction 1: `load %x0, $a, %x4` . . . 12
6.2.2 Instruction 2: `addi %x0, 1, %x3` . . . 13
6.2.3 Instruction 3: `load %x3, $a, %x5` . . . 13
6.2.4 Instruction 4: `add %x4, %x5, %x6` . . . 13
6.2.5 Instruction 5: `end` . . . 13
6.3 Object File . . . 14

7 Instruction Decode Logic – Implementation Theory **14**
7.1 Step 1: Extract the Opcode . . . 14
7.2 Step 2: Determine the Format . . . 14
7.3 Step 3: Extract Fields Based on Format . . . 14
7.4 Step 4: Handle Special Cases for Branches . . . 15

1 ToyRISC ISA Overview

The ToyRISC ISA is a custom RISC instruction set architecture designed for educational purposes. It is inspired by RISC-V (RV32I) and shares many design principles with it. The ISA is clean, minimal, and regular, making it suitable for building a processor simulator from scratch.

1.1 Key Characteristics

- **32-bit fixed-width instructions**: Every instruction is exactly 32 bits, simplifying the fetch and decode logic.
- **32 general-purpose registers**: Named `x0` through `x31`, each 32 bits wide.
- **Load-store architecture**: Only `load` and `store` access memory. All computations occur between registers.
- **30 distinct operations**: Including arithmetic, logical, memory, control flow, and a special `end` instruction.
- **3 instruction formats**: R3-Type, R2I-Type, and RI-Type.
- **5-bit opcode**: Allows for up to $2^5 = 32$ unique opcodes (30 are used, leaving 2 reserved).
- **Word-addressable memory**: $2^{16} = 65536$ addressable words.

2 Instruction Formats – Detailed Encoding

ToyRISC defines three instruction formats. The opcode is always in the most significant 5 bits (bits 31–27). The remaining 27 bits are interpreted differently based on the format.

2.1 R3-Type (Register-Register-Register)

Used by arithmetic/logical instructions that operate on **three registers** (two sources, one destination).

Bits 31–27	**Bits 26–22**	**Bits 21–17**	**Bits 16–12**	**Bits 11–0**
opcode (5)	rs1 (5)	rs2 (5)	rd (5)	unused (12)

- **opcode** (5 bits, [31:27]): Identifies the operation (e.g., 00000 for `add`).
- **rs1** (5 bits, [26:22]): First source register number.
- **rs2** (5 bits, [21:17]): Second source register number.
- **rd** (5 bits, [16:12]): Destination register number.
- **unused** (12 bits, [11:0]): Not used; set to 0.

Example: `add %x3, %x4, %x5` (opcode=00000, rs1=00011, rs2=00100, rd=00101)
Binary encoding:

$$\underbrace{00000}_{\text{add}}\ \underbrace{00011}_{\text{x3}}\ \underbrace{00100}_{\text{x4}}\ \underbrace{00101}_{\text{x5}}\ \underbrace{000000000000}_{\text{unused}}$$

Decimal: Each field contributes to the 32-bit integer value stored in memory.

2.1.1 Bit Extraction Formulas for R3-Type

To decode an R3-Type instruction from a 32-bit integer `instr`:

$$\text{opcode} = (\texttt{instr} \gg 27)\ \&\ \texttt{0x1F} \tag{1}$$
$$\text{rs1} = (\texttt{instr} \gg 22)\ \&\ \texttt{0x1F} \tag{2}$$
$$\text{rs2} = (\texttt{instr} \gg 17)\ \&\ \texttt{0x1F} \tag{3}$$
$$\text{rd} = (\texttt{instr} \gg 12)\ \&\ \texttt{0x1F} \tag{4}$$

In Java: `int opcode = (instr >>> 27) & 0x1F;`

2.2 R2I-Type (Register-Register-Immediate)

Used by arithmetic/logical instructions with an immediate value, memory instructions, and conditional branches.

Bits 31–27	**Bits 26–22**	**Bits 21–17**	**Bits 16–0**
opcode (5)	rs1 (5)	rd (5)	immediate (17)

- **opcode** (5 bits, [31:27]): Identifies the operation.
- **rs1** (5 bits, [26:22]): Source register number.
- **rd** (5 bits, [21:17]): Destination register number.
- **immediate** (17 bits, [16:0]): Signed immediate value (two's complement). Range: -2^{16} to $2^{16} - 1$ (i.e., -65536 to 65535).

Note: For R2I-Type, the ISA specification states that immediate values are placed in `sourceOperand2` in the ParsedProgram representation. This is critical for the Operand Fetch implementation.

2.2.1 Sign Extension of 17-bit Immediate

The 17-bit immediate must be sign-extended to 32 bits before use:

```
int imm17 = instr & 0x1FFFF;   // extract lower 17 bits
// Sign extend: if bit 16 is set, the value is negative
if ((imm17 & (1 << 16)) != 0) {
    imm17 = imm17 | 0xFFFE0000; // fill upper 15 bits with 1s
}
```

Listing 1: Sign extension of 17-bit immediate

Alternatively, using arithmetic shift:

```
int imm17 = (instr << 15) >> 15; // shift left, then arith right
```

2.2.2 Bit Extraction Formulas for R2I-Type

$$\text{opcode} = (\texttt{instr} \gg 27)\ \&\ \texttt{0x1F} \tag{5}$$
$$\text{rs1} = (\texttt{instr} \gg 22)\ \&\ \texttt{0x1F} \tag{6}$$
$$\text{rd} = (\texttt{instr} \gg 17)\ \&\ \texttt{0x1F} \tag{7}$$
$$\text{imm} = \text{signExtend}_{17\to 32}(\texttt{instr}\ \&\ \texttt{0x1FFFF}) \tag{8}$$

2.3 RI-Type (Register-Immediate)

Used by the `jmp` and `end` instructions.

Bits 31–27	Bits 26–22	Bits 21–0
opcode (5)	rd (5)	immediate (22)

- **opcode** (5 bits, [31:27]): Identifies the operation.
- **rd** (5 bits, [26:22]): Destination register (used by `jmp`; unused by `end`).
- **immediate** (22 bits, [21:0]): Signed immediate value. Range: -2^{21} to $2^{21} - 1$.

2.3.1 Bit Extraction Formulas for RI-Type

$$\text{opcode} = (\texttt{instr} \gg 27)\ \&\ \texttt{0x1F} \tag{9}$$
$$\text{rd} = (\texttt{instr} \gg 22)\ \&\ \texttt{0x1F} \tag{10}$$
$$\text{imm} = \text{signExtend}_{22\to 32}(\texttt{instr}\ \&\ \texttt{0x3FFFFF}) \tag{11}$$

2.4 Opcode-to-Format Mapping

Not all opcodes use the same format. Here is the complete mapping:

Opcode	Binary	Format	Operation
0	00000	R3-Type	`add` rd = rs1 + rs2
1	00001	R2I-Type	`addi` rd = rs1 + imm
2	00010	R3-Type	`sub` rd = rs1 - rs2
3	00011	R2I-Type	`subi` rd = rs1 - imm
4	00100	R3-Type	`mul` rd = rs1 × rs2
5	00101	R2I-Type	`muli` rd = rs1 × imm
6	00110	R3-Type	`div` rd = rs1 ÷ rs2
7	00111	R2I-Type	`divi` rd = rs1 ÷ imm
8	01000	R3-Type	`and` rd = rs1 & rs2
9	01001	R2I-Type	`andi` rd = rs1 & imm
10	01010	R3-Type	`or` rd = rs1 | rs2
11	01011	R2I-Type	`ori` rd = rs1 | imm
12	01100	R3-Type	`xor` rd = rs1 ⊕ rs2
13	01101	R2I-Type	`xori` rd = rs1 ⊕ imm
14	01110	R3-Type	`slt` rd = (rs1 < rs2) ? 1 : 0
15	01111	R2I-Type	`slti` rd = (rs1 < imm) ? 1 : 0

16	10000	R3-Type	`sll` rd = rs1 $\ll$ rs2
17	10001	R2I-Type	`slli` rd = rs1 $\ll$ imm
18	10010	R3-Type	`srl` rd = rs1 $\ggg$ rs2 (logical)
19	10011	R2I-Type	`srli` rd = rs1 $\ggg$ imm (logical)
20	10100	R3-Type	`sra` rd = rs1 $\gg$ rs2 (arithmetic)
21	10101	R2I-Type	`srai` rd = rs1 $\gg$ imm (arithmetic)
22	10110	R2I-Type	`load` rd = mem[rs1 + imm]
23	10111	R2I-Type	`store` mem[rd + imm] = rs1
24	11000	RI-Type	`jmp` PC = PC + rd + imm
25	11001	R2I-Type	`beq` if rs1==rd, PC = PC + imm
26	11010	R2I-Type	`bne` if rs1$\neq$rd, PC = PC + imm
27	11011	R2I-Type	`blt` if rs1<rd, PC = PC + imm
28	11100	R2I-Type	`bgt` if rs1>rd, PC = PC + imm
29	11101	RI-Type	`end` terminate execution

Pattern: Even opcodes (0, 2, 4, ..., 20) are R3-Type (register-register). Odd opcodes (1, 3, 5, ..., 21) are R2I-Type (register-immediate). Opcodes 22–28 are memory and branch instructions (all R2I-Type). Opcodes 24 and 29 are RI-Type.

3 Detailed Instruction Semantics

3.1 Arithmetic Instructions

3.1.1 add / addi – Addition

add `%rs1, %rs2, %rd` $\Rightarrow$ `rd = rs1 + rs2`
addi `%rs1, imm, %rd` $\Rightarrow$ `rd = rs1 + imm`

Addition of two 32-bit signed integers. If the result exceeds 32 bits (overflow), the upper 32 bits are stored in `x31`. In Java, integer overflow wraps around naturally (two's complement), so the lower 32 bits are in the result and the overflow can be detected by computing the full 64-bit result using `long`:

```
long result64 = (long)op1 + (long)op2;
int result32 = (int)result64;              // lower 32 bits
int overflow = (int)(result64 >> 32);      // upper 32 bits -> x31
```

3.1.2 sub / subi – Subtraction

sub `%rs1, %rs2, %rd` $\Rightarrow$ `rd = rs1 - rs2`
subi `%rs1, imm, %rd` $\Rightarrow$ `rd = rs1 - imm`

Similar to addition but performs subtraction. Overflow handling is analogous.

Common pattern: `sub %x3, %x3, %x3` sets `x3 = 0`. This is equivalent to `add %x0, %x0, %x3` and is a standard idiom for clearing a register.

3.1.3 mul / muli – Multiplication

mul `%rs1, %rs2, %rd` ⇒ `rd = (rs1 * rs2)[31:0]`, `x31 = (rs1 * rs2)[63:32]`
muli `%rs1, imm, %rd` ⇒ `rd = (rs1 * imm)[31:0]`, `x31 = (rs1 * imm)[63:32]`

Multiplying two 32-bit integers produces a 64-bit result. The lower 32 bits go to `rd`, and the upper 32 bits go to `x31`.

```
long result64 = (long)op1 * (long)op2;
int rd_value = (int)result64;
int x31_value = (int)(result64 >> 32);
```

3.1.4 div / divi – Division

div `%rs1, %rs2, %rd` ⇒ `rd = rs1 / rs2` (quotient), `x31 = rs1 % rs2` (remainder)
divi `%rs1, imm, %rd` ⇒ `rd = rs1 / imm`, `x31 = rs1 % imm`

Integer division. The quotient goes to `rd` and the remainder goes to `x31`. This is heavily used in the palindrome and prime test programs.

```
int rd_value = op1 / op2;   // integer division (truncates)
int x31_value = op1 % op2;  // remainder
```

3.2 Logical Instructions

3.2.1 and / andi – Bitwise AND

`rd = rs1 & rs2` or `rd = rs1 & imm`

Performs bitwise AND. Each bit of the result is 1 only if the corresponding bits of both operands are 1.

Use case: Masking – extracting specific bits from a value. For example, `andi %x3, 1, %x4` extracts the least significant bit of `x3`.

3.2.2 or / ori – Bitwise OR

`rd = rs1 | rs2` or `rd = rs1 | imm`

Performs bitwise OR. Each bit of the result is 1 if either corresponding bit is 1.

Use case: Setting specific bits. For example, `ori %x3, 0x80, %x3` sets bit 7 of `x3`.

3.2.3 xor / xori – Bitwise XOR

`rd = rs1 ⊕ rs2` or `rd = rs1 ⊕ imm`

Performs bitwise exclusive OR. The result bit is 1 if the input bits differ.

Use case: Toggling bits, computing parity, and encryption.

3.3 Comparison Instructions

3.3.1 slt / slti – Set Less Than

```
rd = (rs1 < rs2) ? 1 : 0   or   rd = (rs1 < imm) ? 1 : 0
```

Compares two signed integers. If `rs1` is less than `rs2` (or `imm`), the result is 1; otherwise 0.

Use case: Building complex control flow. For example, checking if a value is within a range.

3.4 Shift Instructions

3.4.1 sll / slli – Shift Left Logical

```
rd = rs1 << rs2 (or << imm)
x31 = bits shifted out
```

Shifts all bits of `rs1` to the left by `rs2` (or `imm`) positions. Zeros are filled in from the right. The bits shifted out (from the left) are stored in `x31`.

Mathematical effect: Left shifting by n is equivalent to multiplication by 2^n.

```
int rd_value = op1 << shiftAmount;
int x31_value = op1 >>> (32 - shiftAmount); // bits shifted out
```

3.4.2 srl / srli – Shift Right Logical

```
rd = rs1 >>> rs2 (logical right shift)
x31 = bits shifted out
```

Shifts all bits to the right. Zeros are filled in from the left (regardless of sign). The bits shifted out (from the right) are stored in `x31`.

In Java, `>>>` is the unsigned/logical right shift operator.

```
int rd_value = op1 >>> shiftAmount;
int x31_value = op1 << (32 - shiftAmount); // bits shifted out
```

3.4.3 sra / srai – Shift Right Arithmetic

```
rd = rs1 >> rs2 (arithmetic right shift, preserves sign)
x31 = bits shifted out
```

Shifts all bits to the right, but fills the left with copies of the **sign bit** (MSB). This preserves the sign of negative numbers.

Mathematical effect: Arithmetic right shifting by n is equivalent to division by 2^n (with rounding toward negative infinity).

In Java, >> is the signed/arithmetic right shift operator.

```
int rd_value = op1 >> shiftAmount;
int x31_value = op1 << (32 - shiftAmount); // bits shifted out
```

3.5 Memory Instructions

3.5.1 load – Load Word from Memory

Format: R2I-Type
Assembly: `load %rs1, imm, %rd`
Semantics: `rd = memory[rs1 + imm]`
Opcode: 10110 (22)

Loads a 32-bit word from main memory at address `rs1 + imm` into register `rd`.
EX stage: Compute effective address = value of rs1 + immediate.
MA stage: Read `memory[effective_address]` and pass the loaded value forward.
RW stage: Write the loaded value to register `rd`.
Example: `load %x0, $a, %x4`

- `rs1 = x0` (value 0)
- `imm = address of label a` (e.g., 0)
- Effective address = 0 + 0 = 0
- `x4 = memory[0]`

3.5.2 store – Store Word to Memory

Format: R2I-Type
Assembly: `store %rs1, imm, %rd`
Semantics: `memory[rd + imm] = rs1`
Opcode: 10111 (23)

Stores the value of register `rs1` into main memory at address `rd + imm`.
Important: Note the asymmetry with `load`. In `store`, the destination register `rd` is used as part of the **address** calculation (not as the register to write to). The value to store comes from `rs1`.
EX stage: Compute effective address = value of rd + immediate.
MA stage: Write value of rs1 to `memory[effective_address]`.
RW stage: No register write needed.

3.6 Control Flow Instructions – Detailed

3.6.1 jmp – Unconditional Jump

Format: RI-Type
Assembly: `jmp label` or `jmp %rd` (one of rd or imm is used, the other is 0)
Semantics: `PC = PC + rd + imm`
Opcode: 11000 (24)

Critical note from ISA spec: In assembly, either `rd` or `imm` is used. In machine code, the unused one is set to zero. In the ParsedProgram, the used one is placed in the `destinationOperand` field.

The new PC is computed as: $\text{PC}_{\text{new}} = \text{PC}_{\text{current}} + \text{rd} + \text{imm}$

Since PC was already incremented during IF, the actual value of "PC" here refers to the PC *before* increment, i.e., the address of the `jmp` instruction itself. This must be handled carefully during implementation.

3.6.2 beq – Branch if Equal

Format: R2I-Type
Assembly: `beq %rs1, %rd, label`
Semantics: If `rs1 == rd`, then `PC = PC + imm`
Opcode: 11001 (25)

ParsedProgram representation: The two registers being compared (`rs1` and `rd` in assembly notation) are placed in `sourceOperand1` and `sourceOperand2`. The label/offset is placed in `destinationOperand`.

3.6.3 bne – Branch if Not Equal

Semantics: If `rs1 != rd`, then `PC = PC + imm`
Opcode: 11010 (26)

3.6.4 blt – Branch if Less Than

Semantics: If `rs1 < rd`, then `PC = PC + imm`
Opcode: 11011 (27)

3.6.5 bgt – Branch if Greater Than

Semantics: If `rs1 > rd`, then `PC = PC + imm`
Opcode: 11100 (28)

3.7 Special Instruction: end

Format: RI-Type
Assembly: `end`
Semantics: Terminate program execution.
Opcode: 11101 (29)
Both `rd` and `imm` fields are unused (set to 0).

When the `end` instruction reaches the RW stage, the simulator calls `Simulator.setSimulationCompl` to break the simulation loop.

4 Object File Format

The ToyRISC assembler converts assembly programs into object files (binary format). The object file is what the simulator reads and loads into memory.

4.1 Structure

The object file contains a sequence of integers (one per line in text representation, or as binary data):

1. The first integer is the **address of the first instruction** (i.e., the value to set PC to). This equals N_d, the number of data words.
2. The subsequent integers are the data and code words, in order. Data words come first (addresses 0 to $N_d - 1$), followed by code words (addresses N_d to $N_t - 1$).

4.2 Loading the Object File

The `loadProgram()` function in `Simulator.java` must:

1. Open the object file.
2. Read the first integer: this is N_d, the start address of the text segment.
3. Read all subsequent integers and store them sequentially in memory starting at address 0.
4. Set `PC =` N_d (address of the first instruction).
5. Set `x0 = 0`, `x1 = 65535`, `x2 = 65535`.

```java
static void loadProgram(String objectFile) {
    // Read the object file
    Scanner scanner = new Scanner(new File(objectFile));

    // First value = PC (address of first instruction)
    int firstInstructionAddress = scanner.nextInt();

    // Load data and instructions into memory
    int address = 0;
```

```
    while (scanner.hasNextInt()) {
        processor.getMainMemory()
            .setWord(address, scanner.nextInt());
        address++;
    }

    // Set PC to the first instruction
    processor.getRegisterFile()
        .setProgramCounter(firstInstructionAddress);

    // Initialize special registers
    processor.getRegisterFile().setValue(0, 0);      // x0 = 0
    processor.getRegisterFile().setValue(1, 65535);   // x1 = SP
    processor.getRegisterFile().setValue(2, 65535);   // x2 = FP
}
```

Listing 2: Loading the program (conceptual)

5 Assembly Language Syntax

5.1 Directives

- `.data`: Marks the beginning of the static data segment.
- `.text`: Marks the beginning of the code segment.

5.2 Labels

Labels are symbolic names for memory addresses. They make assembly programs readable.

- A label is defined by placing a name followed by a colon at the beginning of a line: `loop:`
- Labels in the `.data` section refer to data addresses. `a:` followed by `123` means address 0 contains 123.
- Labels in the `.text` section refer to instruction addresses.
- The special label `main:` indicates the program entry point.
- In instructions, labels are prefixed with `$`: `load %x0, $a, %x4`

5.3 Operand Notation

Prefix	Type	Example
`%`	Register	`%x3` refers to register x3
`$`	Label (address)	`$a` refers to the address named 'a'
(none)	Immediate	`42` is the integer constant 42

5.4 Instruction Ordering

In assembly, the destination operand is always written **last**:

`add %x3, %x4, %x5` means `x5 = x3 + x4` (destination is x5, written last).

This is opposite to some other ISAs (e.g., x86, where the destination is written first).

6 Complete Encoding Example

Let us manually encode the "Adding Two Numbers" program:

```
.data
a:
    123
    234
.text
main:
    load %x0, $a, %x4       ; x4 = memory[x0 + 0] = 123
    addi %x0, 1, %x3        ; x3 = x0 + 1 = 1
    load %x3, $a, %x5       ; x5 = memory[x3 + 0] = memory[1] = 234
    add %x4, %x5, %x6       ; x6 = x4 + x5 = 123 + 234 = 357
    end
```

Listing 3: Adding Two Numbers

6.1 Data Segment

- Address 0: 123
- Address 1: 234

$N_d = 2$ (2 data words). Instructions start at address 2.

6.2 Instruction Encoding

6.2.1 Instruction 1: `load %x0, $a, %x4`

- Opcode: `load` = 10110 = 22
- Format: R2I-Type
- rs1 = x0 = 00000
- rd = x4 = 00100
- imm = address of `a` = 0 = 00000000000000000 (17 bits)

Binary: `10110 00000 00100 00000000000000000`
As 32-bit integer: $10110\ 00000\ 00100\ 0\ldots0_2 = (22 \ll 27)|(0 \ll 22)|(4 \ll 17)|0$
$= 22 \times 2^{27} + 0 + 4 \times 2^{17} + 0 = 2952790016 + 524288 = -1341653504$ (in signed 32-bit)

6.2.2 Instruction 2: `addi %x0, 1, %x3`

- Opcode: `addi` = 00001 = 1
- Format: R2I-Type
- rs1 = x0 = 00000
- rd = x3 = 00011
- imm = 1 = 00000000000000001

Binary: `00001 00000 00011 00000000000000001`

6.2.3 Instruction 3: `load %x3, $a, %x5`

- Opcode: `load` = 10110 = 22
- rs1 = x3 = 00011
- rd = x5 = 00101
- imm = 0

6.2.4 Instruction 4: `add %x4, %x5, %x6`

- Opcode: `add` = 00000 = 0
- Format: R3-Type
- rs1 = x4 = 00100
- rs2 = x5 = 00101
- rd = x6 = 00110
- unused = 000000000000

Binary: `00000 00100 00101 00110 000000000000`

6.2.5 Instruction 5: `end`

- Opcode: `end` = 11101 = 29
- Format: RI-Type
- rd = 0, imm = 0

Binary: `11101 00000 00000000000000000000`

6.3 Object File

The complete object file would contain:

```
2       (first instruction address = Nd)
123     (data: memory[0])
234     (data: memory[1])
-1341653504   (instruction: load at memory[2])
...     (instruction: addi at memory[3])
...     (instruction: load at memory[4])
...     (instruction: add at memory[5])
...     (instruction: end at memory[6])
```

7 Instruction Decode Logic – Implementation Theory

The Operand Fetch (OF) stage must decode the 32-bit instruction and extract all necessary fields. Here is a systematic approach:

7.1 Step 1: Extract the Opcode

```
int instruction = IF_OF_Latch.getInstruction();
int opcode = (instruction >>> 27) & 0x1F;
```

7.2 Step 2: Determine the Format

```
// R3-Type: opcodes 0,2,4,6,8,10,12,14,16,18,20
// R2I-Type: opcodes 1,3,5,7,9,11,13,15,17,19,21,22,23,25,26,27,28
// RI-Type: opcodes 24, 29
boolean isR3Type = (opcode <= 21 && opcode % 2 == 0);
boolean isRIType = (opcode == 24 || opcode == 29);
boolean isR2IType = !isR3Type && !isRIType;
```

7.3 Step 3: Extract Fields Based on Format

```
if (isR3Type) {
    int rs1 = (instruction >>> 22) & 0x1F;
    int rs2 = (instruction >>> 17) & 0x1F;
    int rd  = (instruction >>> 12) & 0x1F;
    // Read register values
    int val_rs1 = registerFile.getValue(rs1);
    int val_rs2 = registerFile.getValue(rs2);
} else if (isR2IType) {
    int rs1 = (instruction >>> 22) & 0x1F;
    int rd  = (instruction >>> 17) & 0x1F;
    int imm = instruction & 0x1FFFF; // 17-bit immediate
    // Sign extend
    if ((imm & (1 << 16)) != 0)
        imm = imm | 0xFFFE0000;
```

```
    int val_rs1 = registerFile.getValue(rs1);
} else { // RI-Type
    int rd  = (instruction >>> 22) & 0x1F;
    int imm = instruction & 0x3FFFFF; // 22-bit immediate
    // Sign extend
    if ((imm & (1 << 21)) != 0)
        imm = imm | 0xFFC00000;
}
```

7.4 Step 4: Handle Special Cases for Branches

For conditional branches (`beq`, `bne`, `blt`, `bgt`), the ISA specification says:

> "In ParsedProgram, for conditional branches, the two registers that are compared are placed in sourceOperand1 and sourceOperand2. The imm value is placed in destinationOperand."

This means:

- `sourceOperand1` = value of rs1
- `sourceOperand2` = value of rd (not the immediate!)
- `destinationOperand` = the immediate offset

The OF/EX latch must be designed to carry all this information.

End of Part 2. Continue to Part 3 for complete simulator architecture, code walkthrough, test program analysis, and implementation guide.